

Assignment: 4

Title:- Design a base class shape with two double type values and member functions to input the data and compute_area() for calculating area of figure. Derive two classes' triangle and rectangle. Make compute_area() as abstract function and redefine this function in the derived class to suit their requirements. Write a program that accepts dimensions of triangle/rectangle and display calculated area. Implement dynamic binding for given case study.

Objectives :-

- 1) To understand the concept of abstract
- 2) To implement dynamic binding or run time polymorphism

Theory:-

Dynamic Binding

Connecting a method call to the method body is known as binding.

There are two types of binding

- Static Binding (also known as Early Binding): The binding which can be resolved at compile time by the compiler is known as static or early binding. The binding of all the static, private, and final methods is done at compile-time.
- Dynamic Binding (also known as Late Binding). In Dynamic binding compiler doesn't decide the method to be called. Overriding is a perfect example of dynamic binding. In overriding both parent and child classes have the same method.
In dynamic binding, the method call is bonded to the method body at runtime. This is also known as late binding. This is done using instance methods.

Example of dynamic binding

```
class Shape {
    public void draw() {
        System.out.println("Drawing a shape");
    }
}
class Circle extends Shape {
    @Override
    public void draw() {
        System.out.println("Drawing a circle");
    }
}
class Rectangle extends Shape {
    @Override
    public void draw() {
        System.out.println("Drawing a rectangle");
    }
}
public class Main {
    public static void main(String[] args) {
```

```

        Shape shape1 = new Circle();
        Shape shape2 = new Rectangle();
        shape1.draw(); // Output: Drawing a circle
        shape2.draw(); // Output: Drawing a rectangle
    }
}

```

In the main method, we create two shape objects: shape1 of type Circle and shape2 of type Rectangle. Although the reference type is Shape, the actual objects being referred to are Circle and Rectangle.

When we call the draw() method on shape1 and shape2, the JVM dynamically binds the appropriate version of the draw() method at runtime based on the actual type of the object.

Abstract class in Java

A class which is declared as abstract is known as an abstract class. It can have abstract and non-abstract methods. It needs to be extended and its method implemented.

We cannot be instantiated the abstract class in Java directly. Instead, we can subclass the abstract class. When we use an abstract class as a subclass, the abstract class method implementation becomes available to all of its parent classes.

The Syntax for Abstract Class

To declare an abstract class, we use the access modifier first, then the "abstract" keyword, and the class name shown below.

```

<Access_Modifier> abstract class <Class_Name> {

//Data_Members;

//Statements;

//Methods;

}

```

Steps :

1. Start
2. Create an abstract class named shape that contains two double type numbers and an empty method named compute_area().
3. Provide two classes named rectangle and triangle such that each one of the classes extends the class Shape.
4. Each of the inherited class from shape class should provide the implementation for the method compute_area().
5. Get the input and calculate the area of rectangle and triangle.
6. In the fourth separate class, create the objects for the two inherited classes and invoke the methods and display the area values of the different shapes.
7. Stop.

Input:

length and breadth of rectangle
base and height of triangle

Output:

area of rectangle area of circle

Test case or Validation:

Different values for length and breadth of rectangle and base and height of triangle.

Result:- Area of Circle and area of Rectangle

Conclusion :- Thus studied the concept of dynamic binding or runtime polymorphism using java.

Frequently asked Question

1. What is Abstraction?
2. Rules to Declare Abstract Class.
3. What are different way to achieve abstraction in Java
4. What is static and dynamic binding?
5. What are features of of Abstract Class

(write answer for the above questions) & Attached code with output